

Методы оптимизации и исследование операций · лекция 1

Организационное

- После лекции преподаватель объявила небольшой **тест по материалу лекции**. Проходит в **Moodle** сразу после лекции; преподаватель открыла тест и изменила время.
 - **Завтра** — продолжение практических занятий у **первой, второй и третьей групп** (по расписанию).
 - Других организационных указаний (конкретных дедлайнов, формы сдачи лабораторных/курсовых и т. п.) в лекции **не упоминалось**.
-

Тема лекции

Динамическое программирование (лекция №6 по нумерации преподавателя).

План лекции

1. Определение динамического программирования и история появления.
 2. Принцип оптимальности Беллмана.
 3. Постановка задачи, функция Беллмана, функциональные уравнения.
 4. Общая схема решения: условная и безусловная оптимизация.
 5. Примеры задач:
 - наивыгоднейший путь между двумя пунктами;
 - замена оборудования;
 - оптимальное распределение ресурсов.
 6. Рекурсия и динамическое программирование: сравнение.
 7. Жадные алгоритмы. Алгоритм Дейкстры.
 8. Достоинства, недостатки и практическое применение методов.
-

Основные определения

Динамическое программирование (ДП) — раздел математического программирования, предназначенный для решения **многошаговых задач**, где процесс принятия решений разбивается на несколько этапов (шагов). Впервые использовано **Беллманом в 40-х годах** для описания процесса нахождения решений, в котором ответ на одну задачу получается только после решения предыдущей.

Многошаговый процесс — процесс принятия решений, в котором управление и состояние системы рассматриваются последовательно, шаг за шагом.

Принцип оптимальности Беллмана (сформулирован в **1953 году**):

Каково бы ни было состояние системы, на каждом этапе принимается такое решение, которое обеспечивает оптимальность с данного этапа до конца процесса.

Функция Беллмана — целевая функция, характеризующая выигрыш на отдельном шаге процесса.

Функциональные уравнения Беллмана — математическая формулировка принципа оптимальности, позволяющая вычислять оптимальное управление на каждом шаге.

Условно оптимальное управление — управление, найденное для каждого шага при движении от конца процесса к началу.

Оптимальное управление — управление (X^*), при котором достигается максимум (или минимум) целевой функции; состоит из совокупности оптимальных шаговых управлений.

Рекуррентные соотношения — основной метод динамического программирования, связывающий оптимальные значения на соседних шагах.

Жадные алгоритмы — алгоритмы, которые на каждом этапе принимают локально оптимальное решение, надеясь, что в итоге решение окажется глобально оптимальным.

Разбор материала лекции

1. Зачем нужно динамическое программирование

В линейном программировании, в отличие от динамического, существует универсальный метод — симплекс-метод. В динамическом программировании **универсального метода нет**, однако с его помощью можно сложную задачу со многими переменными свести к последовательности задач с малым количеством переменных.

ДП применяется для задач, которые можно представить как **управляемый многошаговый процесс**: на каждом шаге выбирается некоторое решение (управление), которое влияет на ход и исход операции.

2. Принцип оптимальности

Суть принципа оптимальности:

На каждом этапе необходимо принимать решение и просматривать его последствия **до конца процесса**.

То есть управление строится так, чтобы оптимальность обеспечивалась с текущего шага и до конца, а не только на данном шаге.

3. Постановка задачи. Функция Беллмана

Пусть система в начальный момент находится в состоянии (S_0).

Под действием управления (x_1) она переходит в состояние (S_1), затем под действием (x_2) — в состояние (S_2), и так далее:

$[S_0 \xrightarrow{x_1} S_1 \xrightarrow{x_2} S_2 \xrightarrow{x_3} \dots \xrightarrow{x_n} S_n]$

На каждом шаге вычисляется **целевая функция** (функция Беллмана), которая зависит от управления и состояния.

Задача динамического программирования:

подобрать такое управление (X), которое переводит систему из начального состояния (S_0) в конечное за (n) шагов и обеспечивает **максимум** (или минимум) целевой функции.

Показатель эффективности операции — **выигрыш** (W). Если выигрыши на отдельных шагах равны ($w_1, w_2, \dots, w_m$), то суммарный выигрыш:

$$[W = w_1 + w_2 + \dots + w_m]$$

Требуется найти управление (X), при котором (W) достигает максимума. Оптимальное управление обозначается (X^*) .

4. Функциональные уравнения Беллмана

Принцип оптимальности можно записать в виде функциональных уравнений. Для (n)-го шага:

$$[F_n(S_{n-1}) = \max_{\{x_n\}} \left[w_n(S_{n-1}, x_n) + F_{n-1}(S_n) \right]]$$

где $(F_n(S_{n-1}))$ — максимальный выигрыш за (n) шагов, начиная с состояния (S_{n-1}) .

Аналогично для ((n-1))-го шага и всех остальных. Решение строится последовательно от последнего шага к первому.

5. Общая схема решения

Процесс оптимизации проходит **дважды**:

1. **Первый проход — от конца к началу** (условная оптимизация):
для каждого шага находят **условно оптимальное управление** и **условно оптимальный выигрыш**.
2. **Второй проход — от начала к концу** (безусловная оптимизация):
используя найденные условно оптимальные управления, строят **безусловное оптимальное управление** для всего процесса.

6. Свойства задач, применимых для ДП

- Задача допускает интерпретацию как (n)-шаговый процесс принятия решений.
- Задача определена для **любого числа шагов**, и её структура не зависит от их числа.
- Должно быть задано множество параметров, от которых зависит оптимальное значение.
- Выбор решения на (k)-м шаге не должен влиять на предыдущие решения, кроме пересчёта переменных.

Примеры задач динамического программирования

6.1. Наивыгоднейший путь между двумя пунктами

Даны пункты (A) и (B). Двигаться можно только **строго на восток** или **строго на север**. Расстояние от (A) до (B) делится: на восток — на 7 частей, на север — на 5 частей. Любой путь состоит из 12 отрезков.

На каждом отрезке указана **стоимость** перехода. Нужно найти путь с **минимальной суммарной стоимостью**.

Решение методом ДП:

1. **Условная оптимизация** — движение от (B) к (A) (от конца к началу).
 - Для последнего шага из точки (B_1) путь на восток стоит 10, из (B_2) путь на север — 14. Эти числа записываются в кружках.
 - Затем рассматриваются точки предпоследнего шага (C_1, C_2, C_3) .
 - Для (C_1) : путь на восток — $11 + 10 = 21$.

- Для (C_2): можно идти на восток или на север, выбирается меньшее: $13 + 10 = 23$; другой вариант $14 + 14 = 28$, поэтому берём 23.

- Для (C_3): $14 + 8 = 22$.

◦ Так продолжают до точки (A). В результате получается условный оптимальный выигрыш ($W^* = 118$).

2. **Безусловная оптимизация** — движение от (A) к (B) по стрелкам, полученным при условной оптимизации. Получаем оптимальную траекторию.

3. **Сравнение с «наивным» способом**: если на каждом шаге выбирать самое выгодное локальное направление (без учёта последствий), получается траектория со стоимостью ($W = 128$), что **больше** оптимальной ($W^* = 118$).

Вывод: метод ДП даёт лучший результат, чем локально «жадный» выбор.

6.2. Задача о замене оборудования

Обозначения:

- $(r(t))$ — стоимость продукции, произведённой за год на оборудовании возраста (t) лет;
- $(u(t))$ — ежегодные затраты на обслуживание оборудования возраста (t) ;
- $(c(t))$ — остаточная стоимость оборудования возраста (t) ;
- (P) — покупная цена нового оборудования;
- $(F_N(t))$ — максимальный доход от оборудования возраста (t) за оставшиеся (N) лет.

Возраст $(t=0)$ соответствует новому оборудованию.

На каждом этапе принимается решение: **сохранить** оборудование или **заменить** его. Используются функциональные уравнения, основанные на принципе оптимальности. Для последнего шага, когда возраст оборудования достигает (N) , слагаемые, связанные с будущими периодами, исключаются.

Вычисления продолжаются до тех пор, пока не станет выгоднее заменить оборудование, чем продолжать его эксплуатировать. Результаты сводятся в таблицу; момент замены отмечается звёздочкой.

В разобранный задаче оптимальный цикл замены оборудования составил **4 года**, то есть для получения максимальной прибыли за 12-этапный процесс оборудование следует заменять каждые 4 года.

6.3. Оптимальное распределение ресурсов

Имеется некоторое количество ресурсов (x) , которые нужно распределить между (n) предприятиями, чтобы получить **максимальную суммарную эффективность**.

Обозначения:

- (x) — количество ресурсов;
- $(g_i(x))$ — функция полезности (доход) от (i) -го предприятия;
- $(f_k(x))$ — максимальный доход от первых (k) предприятий.

Решение основано на рекуррентном соотношении, связывающем доход от (k) предприятий с доходом от $(k-1)$ предприятий:

$$[f_k(x) = \max_{0 \leq x_k \leq x} \{ g_k(x_k) + f_{k-1}(x - x_k) \}]$$

Пример. Совет директоров выделяет **120 млн рублей** на расширение мощностей четырёх предприятий, дискретность — **20 млн рублей**. Дана таблица прироста выпуска продукции на каждом предприятии в зависимости от вложенных средств. Дополнительное условие: на одно предприятие — не более одной инвестиции.

Решение разбивается на 4 этапа (по числу предприятий). Используя рекуррентные соотношения для каждого этапа, получаем:

- оптимально выделить **второму, третьему и четвёртому предприятиям по 40 млн рублей**;
- максимальный прирост выпуска продукции составит **64** единицы.

Рекурсия и динамическое программирование: сравнение

Характеристика	Рекурсия	Динамическое программирование
Подход	Сверху вниз: задача разбивается на меньшие подзадачи, функция вызывает саму себя	Снизу вверх: результаты подзадач сохраняются и используются для построения решения
Определение	Функция вызывает себя до выполнения базового условия	Разбиение на подзадачи с сохранением результатов, чтобы избежать повторных вычислений
Производительность	Медленнее, так как возможны избыточные повторные вычисления	Быстрее, так как подзадачи решаются один раз
Использование памяти	Больше памяти (стек вызовов)	Требуется дополнительная память для хранения промежуточных результатов

Жадные алгоритмы и алгоритм Дейкстры

Классическая задача о рюкзаке

Имеется рюкзак заданной вместимости и (n) предметов с определёнными весами и ценностями. Нужно выбрать набор предметов так, чтобы ценность была **максимальной**, а вес не превышал вместимость.

Способы решения:

- полный перебор;
- метод ветвей и границ;
- **жадные алгоритмы**.

Жадные алгоритмы

Жадный алгоритм — это алгоритм, который на каждом шаге принимает **локально оптимальное решение**, предполагая, что конечное решение также будет оптимальным.

Примеры жадных алгоритмов:

- алгоритм Дейкстры (поиск кратчайшего пути);

- алгоритм Флойда;
- алгоритмы для задачи о минимальном покрывающем дереве;
- алгоритмы сжатия информации (например, Хаффмана).

Алгоритм Дейкстры

Применяется для поиска **кратчайшего пути в графе с неотрицательными весами рёбер**.

Шаги алгоритма:

1. Начальная вершина получает метку **0**, все остальные — большое число (или бесконечность).
2. Обрабатываемая вершина — та, расстояние до которой минимально.
3. Для каждого соседа обрабатываемой вершины вычисляется длина пути: метка текущей вершины + вес ребра.
4. Если полученное значение меньше текущей метки соседа, метка соседа обновляется.
5. После прохода по всем соседям текущая вершина помечается как **обработанная**.
6. Выбирается следующая необработанная вершина с наименьшей меткой, и процесс повторяется, пока все вершины не будут обработаны.

Пример на графе из 5 вершин:

- Начальная вершина (V_1): метка 0.
- Рёбра из (V_1): к (V_2) — 7, к (V_3) — 9, к (V_5) — 4.
Метки: ($V_2 = 7$), ($V_3 = 9$), ($V_5 = 4$).
- Выбираем (V_5) (метка 4). Ребро к (V_3) вес 2: ($4+2=6 < 9$), обновляем метку ($V_3 = 6$).
- Выбираем (V_3) (метка 6). Рёбра: к (V_2) вес 8: ($6+8=14 > 7$), не меняем; к (V_4) вес 11: ($6+11=17$), метка ($V_4 = 17$).
- Выбираем (V_2) (метка 7). Ребро к (V_4) вес 12: ($7+12=19 > 17$), не меняем.
- Все вершины обработаны. Кратчайшие расстояния найдены.

Преимущества жадных алгоритмов

- **Простота** — легко понять и реализовать.
- **Эффективность** — могут работать быстрее других алгоритмов.
- **Меньше памяти** — не нужно хранить все возможные решения.
- Подходят для систем с ограниченными ресурсами.

Недостатки жадных алгоритмов

- Не всегда дают **глобально оптимальное** решение.
- Успешность зависит от структуры конкретной задачи.
- Ограниченность применения: в некоторых случаях нужны более сложные алгоритмы (динамическое программирование, метод ветвей и границ).

Практическое применение жадных алгоритмов

- **Маршрутизация** — оптимизация маршрутов доставки товаров, поиск кратчайшего пути.
 - **Задачи планирования** — распределение задач и ресурсов, составление расписаний.
 - **Сжатие данных** — алгоритм Хаффмана для построения оптимальных кодов.
-

Достоинства и недостатки динамического программирования

Достоинства

- Хорошо приспособлен к **дискретным задачам**.
- Применим при любом способе задания функций и любом допустимом множестве состояний и управлений (в отличие от классических методов).
- Даёт возможность **анализа чувствительности** к изменению исходных данных.

Недостатки

- **Нет универсального алгоритма**, пригодного для всех задач.
 - При больших размерностях задачи требуются **значительное время и ресурсы компьютера**.
-

Выводы

- Динамическое программирование основано на **принципе оптимальности Беллмана**: на каждом шаге нужно принимать решение так, чтобы обеспечить оптимальность до конца процесса.
 - Сложная задача разбивается на **шаги**; на каждом шаге решается простая подзадача оптимизации.
 - Решение осуществляется **дважды**: от конца к началу (условная оптимизация) и от начала к концу (безусловная оптимизация).
 - Динамическое программирование эффективно для задач:
 - о кратчайшем пути;
 - о замене оборудования;
 - о распределении ресурсов;
 - о рюкзаке.
 - **Жадные алгоритмы** (например, алгоритм Дейкстры) — простой и быстрый способ решения некоторых оптимизационных задач, но они не всегда гарантируют глобальный оптимум.
-

Вопросы к зачёту

1. Что такое динамическое программирование? Когда и кем оно было впервые использовано?
2. Сформулируйте принцип оптимальности Беллмана.
3. Что такое многошаговый процесс? Приведите примеры.
4. Что такое функция Беллмана и функциональные уравнения Беллмана?
5. Опишите общую схему решения задач динамического программирования.
6. В чём различие условной и безусловной оптимизации?
7. Какими свойствами должны обладать задачи для применимости динамического программирования?
8. Как решается задача о наивыгоднейшем пути между пунктами?
9. Как решается задача о замене оборудования?
10. Как решается задача распределения ресурсов?
11. Каковы достоинства и недостатки динамического программирования?
12. Чем рекурсия отличается от динамического программирования?
13. Что такое жадные алгоритмы? Приведите примеры.
14. Опишите алгоритм Дейкстры. Для каких графов он применим?
15. Назовите практические применения жадных алгоритмов.

16. Что такое задача о рюкзаке и какими способами её можно решить?